✱ Chapter - 02 ✱
✱ Combinational Circuits ✱

- Types of Digital Circuits

① Combinational Circuit	Sequential Circuit
- o/p depends only on present i/p only	- o/p depends on pres i/p as well as pas i/p values
- There is no memory	- There is memory
- There is no feedback	- There is feedback
eg. H.A, F.A, H.S, F.S, mux, etc	- eg F/F, counter, shif register, etc

- How to Design combinational Circuits
① Identify the no. of inputs lines and no. of o/p lines required to design the ckt
② Write down the table bet" i/p lines and o/p lines
③ Write down o/ps using SOP or POS in terms of i/p lines
④ Now simplyfy o/ps using boolean theorem or K-Map
⑤ Now implement every o/p using logic gates and that's how we finally designed the ckt

Que Design the combinational ckt having 3-i/p lines and o/p is '1' if i/p is divided by '2'.

→ i/p lines → '3' → (A, B, C)
o/p lines → '1' → f

0, 2, 4, 6, 8 are divided by 2 so o/p is '1'

A	B	C	f
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Scanned with OKEN Scanner

$F(A,B,C) = \Sigma(0,2,4,6)$

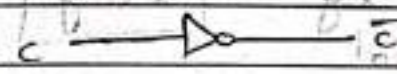
$= \bar{A}\bar{B}\bar{C} + \bar{A}B\bar{C} + A\bar{B}\bar{C} + AB\bar{C}$

$= \bar{B}\bar{C}(\bar{A}+A) + B\bar{C}(\bar{A}+A)$

$= \bar{B}\bar{C} + B\bar{C}$

$= \bar{C}(\bar{B}+B)$

$\boxed{f(A,B,C) = \bar{C}}$



- decimal No system:

  786
  234
  1 1
  1020

- Octal No system

  $(234)_8$
  $(172)_8$
  $(426)_8$

- Binary → 1 0 10
  1 1 0
  1 0 0 0

  A = $a_3\ a_2\ a_1\ a_0$
  B = $b_3\ b_2\ b_1\ b_0$
  C_0 C_1
  C_4 S_3 S_2 S_1 S_0

* Half Adder :-

- 2 i/p lines → [x, y] → corresponding bits of the two no. being added
  $x \to a_0$, $x \to a_1$, $x \to a_2$
  $y \to b_0$, $y \to b_1$, $y \to b_2$
- o/p lines → 2 o/p lines → Sum o/p S
  → carry o/p C

  carry o/p will be '1' if addition is greater than or equal to base otherwise '0'.

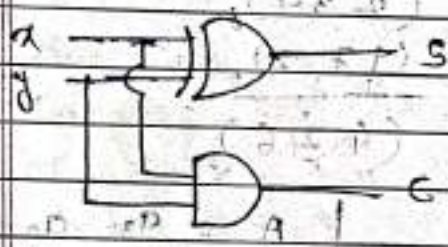
I/p lines		O/p lines	
x	y	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$S(x,y) = \Sigma(1,2) = \bar{x}y + x\bar{y} = \boxed{x \oplus y}$

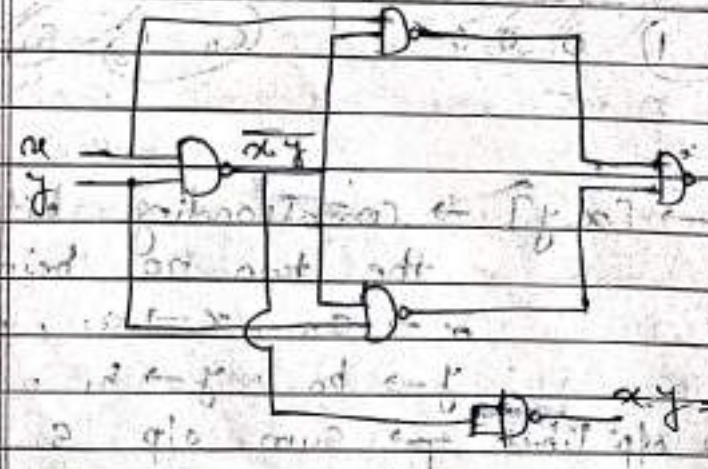
→ Non-self dual

$C(x,y) = \Sigma(3) = \boxed{x \cdot y}$

→ Non-self dual



- Using NAND Gates



- Using NOR Gates → Same as above just replace ~~NOR~~ by NAND by NOR.
- To implement H.A 5 [2-i/p NAND] or 5 [2-i/p NOR] gates are required

* Full Adder :-

- i/p lines → 3-i/p lines (x, y, z)
  x, y → corresponding bits of the two numbers being added
  z → carry o/p from the just previous addition

x → a_1
y → b_1
z → C_1 → carry o/p from (a_1, b_0)
x → a_2
y → b_2
z → C_2 → carry o/p from (a_1, b_1) position

- o/p lines → 2 o/p lines → Sum o/p S
  → Carry o/p C
  - will be '1' if addition is greater than or equal to base = 2
  otherwise '0'.

- Truth Table

I/p lines			o/p lines	
x	y	z	S	C
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$S(x,y,z) = \Sigma(1,2,4,7)$

$S = x \oplus y \oplus z$

→ Self Dual

$C(x,y,z) = \Sigma(3,5,6,7)$

→ Self Dual

$= \bar{x}yz + x\bar{y}z + xy\bar{z} + xyz$

$C = xy + yz + xz$

Carry o/p → is majority i/p function / majority function

$S = x \oplus y \oplus z$

$C = xy + (x \oplus y)z = xy + yz + xz$

- Carry o/p Hidden Tricks

① $C(1,1,0) = 1 \longrightarrow$ Majority

$C(0,1,0) = 0$

$C(x,x,y) = x$

② $C(x,x,y) = x\bar{x} + \bar{x}y + xy$

$= y$

③ $C(x,y,z) = \bar{x}yz + x\bar{y}z + xy\bar{z} + xyz$

$= xy + z[\bar{x}y + x\bar{y}]$

$= xy + z(x \oplus y)$

④ $xy + yz + xz = xy + (x \oplus y)z$

$x + y + z \rightarrow xy + (x \oplus y)z$

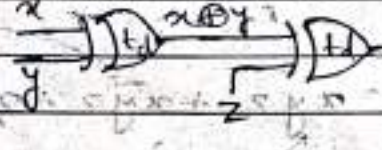
$y + z + x \rightarrow yz + (y \oplus z)x$

$z + x + y \longrightarrow xz + (x \oplus z)y$

⑤ $xy \oplus yz \oplus xz = xy + yz + xz$

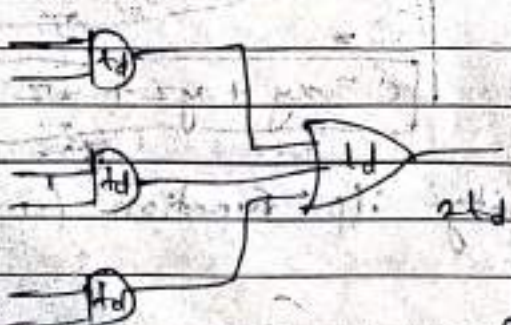
⑥ $xy \oplus [(x \oplus y)z] = xy + (x \oplus y)z$

- Implementation using Gates



$x \oplus y \oplus z = S$

2 td

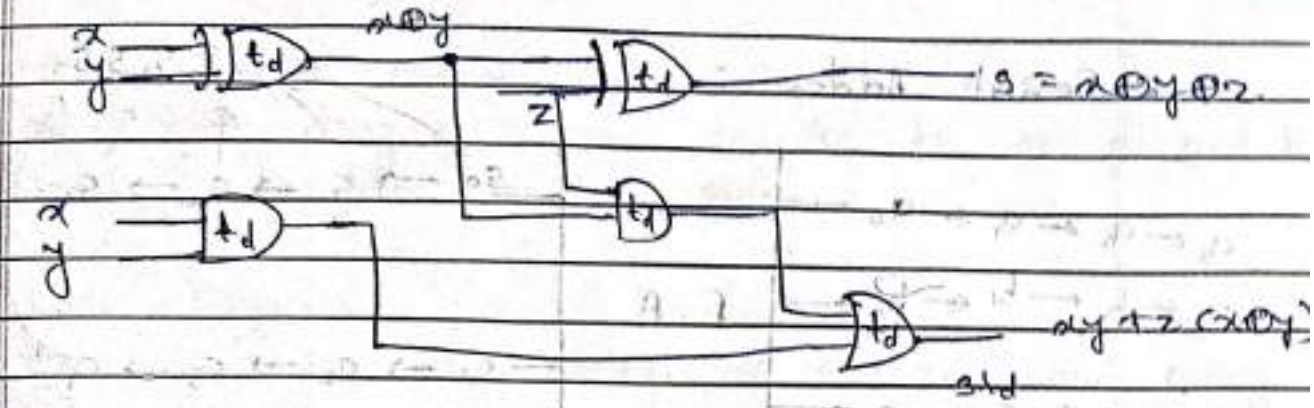


$xy + yz + xz = C$

2 td

Overall delay = 2 td

because they runs parallel



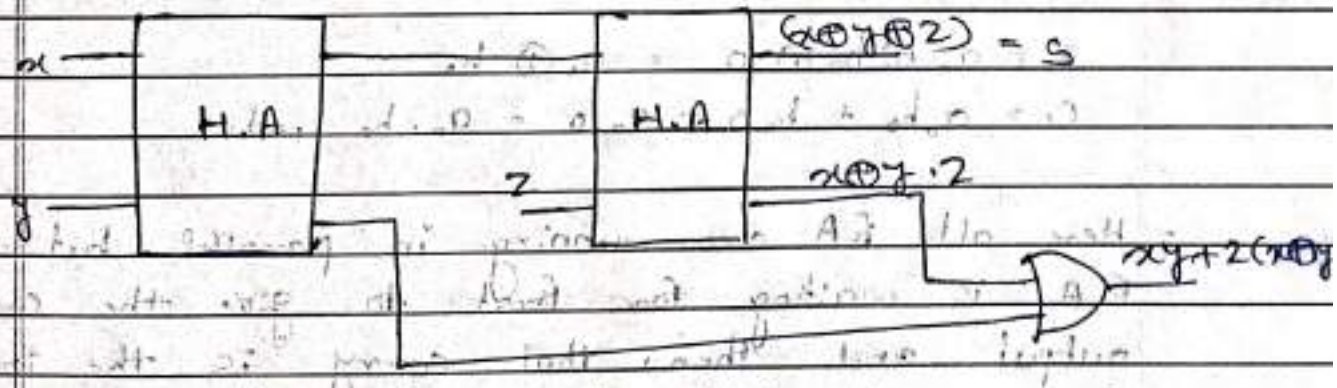
overall delay = $3t_d$

- Note :- To implement full adder 9 [2 -i/p NAND] gates or 9 [2 -i/p NOR] gates are required

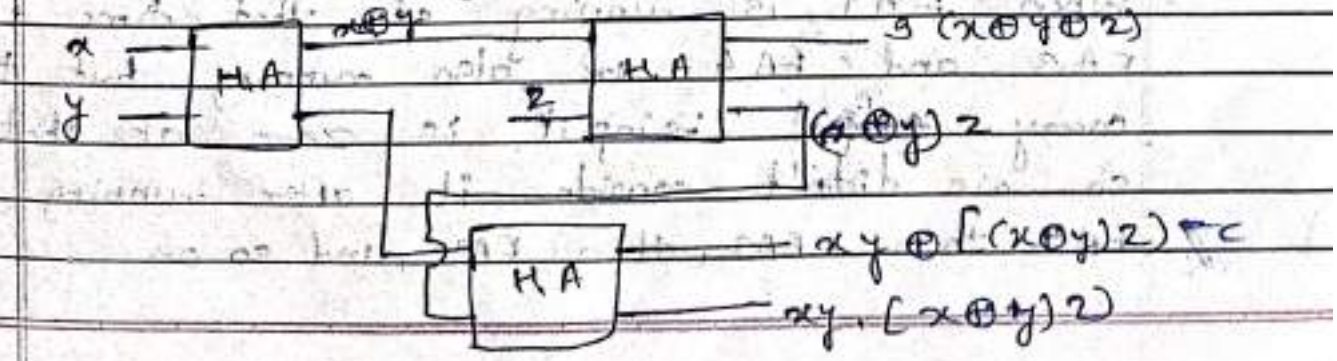
- Full Adder using H.A and OR gate

$S(x,y,z) = x \oplus y \oplus z$

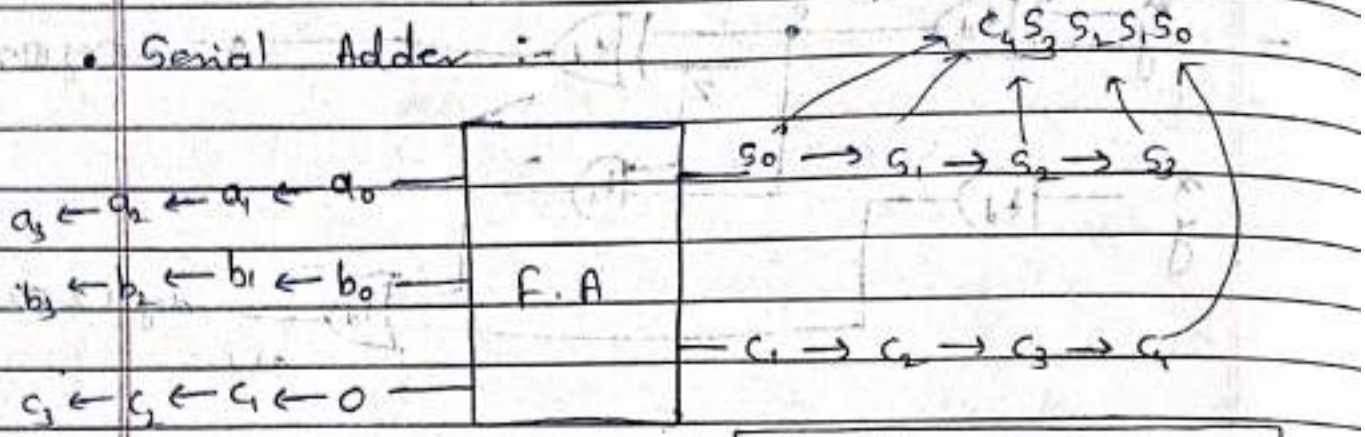
$C(x,y,z) = xy + (x \oplus y)z$



- Full Adder using only H.A



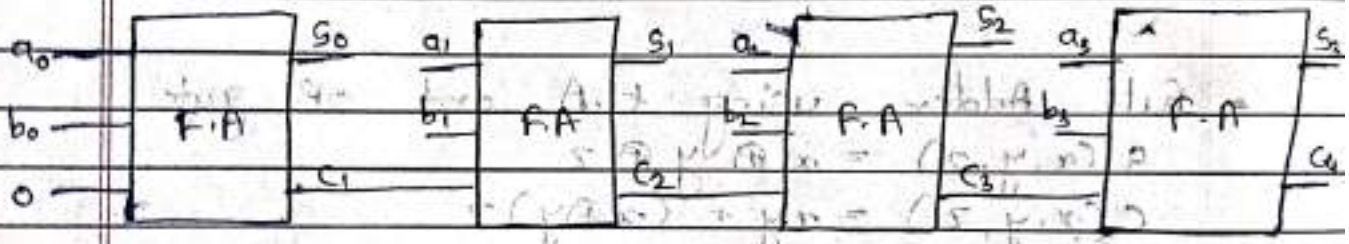
- Serial Adder :-



Result = $C_4 S_3 S_2 S_1 S_0$

- Because of Serial operation the delay in producing the final result is very high that why we don't prefer using Serial adder.

- Parallel Adder :-



$S_0 = a_0 \oplus b_0 \oplus 0 = a_0 \oplus b_0$

$C_1 = a_0 b_0 + b_0 0 + a_0 0 = a_0 \cdot b_0$

- Here all FA are running in parallel but second FA is waiting for first to give the carry output and then that carry is the input for second FA, so on.
- When F.A 1 is running at that time FA2, FA3 and FA4 are also running but their carry ~~output~~ input is zero at that time so we didn't consider it after running of FA1 then FA2, then FA3, and so on.

- It is also called Ripple carry adder
- In above diagram there are 4-bits of a and b so it 4-bit parallel adder

- Case - Each F.A has sum delay of t_s = 2ns and carry delay of t_c = 1.5ns

$$A = \overset{4.5ns}{a_3}\ \overset{3ns}{a_2}\ \overset{1.5ns}{a_1}\ a_0$$

$$B = b_3\ b_2\ b_1\ b_0$$

$$c_3\ c_2\ c_1\ 0$$

$$S_3\ S_2\ S_1\ S_0$$

6ns 6.5ns 5ns 3.5 2ns

Overall delay = 6.5 ns

- Case 2 :- Each F.A has sum delay of t_s = 1ns and carry delay t_c = 1.5ns.

$$A = \overset{4.5ns}{a_3}\ \overset{3ns}{a_2}\ \overset{1.5ns}{a_1}\ a_0$$

$$B = b_3\ b_2\ b_1\ b_0$$

$$c_3\ c_2\ c_1\ 0$$

$$C_4\ S_3\ S_2\ S_1\ S_0$$

6ns 5.5ns 4ns 2.5ns 1ns

overall delay T_d = 6 ns

Scanned with OKEN Scanner

Imp * for n-bit parallel adder

Overall delay = $T_d = (n-1)\,t_c + \max(t_i, t_f)$

- Worst case delay [maximum delay in generating the result].
- Time taken by parallel adder to complete one addition [one result generation] is equal to T_d in worst case scenario.
- Above formula is valid only if sum delay of all F.A is same and carry delay of all F.A is same, otherwise we need to find overall delay by main method on above page.

• How many additions a parallel adder [n bit] can perform in 1 sec

$$N = \frac{1\ \text{sec}}{T_d\ (\text{sec})}$$

* Subtraction

① Decimal Number System

```
   7  8  G
   2  9  8      → Hex base is 16 so G becomes 16
borrow → 1  1
   0  4  8  8  → Result
```

Hex base is 8 so that 0 becomes 8 when we take borrow

② Binary Number System

```
A =  1  1  0  0
B =  0  1  1  0
     1  1  0
  0  0  1  1  0  → Result
```

Result is +ve +[0110]

2's complement
Start from right ↓
upto coming 1 ← 100110
write same but after that flip ALL bits

B = 0 1 1 0
A = 1 1 0 0

1 0 1 0 → Result

result is -ve

↳ -[magnitude]
= -[2's complement of result)
= -[0110]
= (-6)$_{10}$

* Half Subtractor :-

- i/p lines → 2 i/p lines (x, y) → corresponding bits of the two numbers being subtracted

$x \to a_0$, $x \to a_1$, $x \to a_2$, etc.
$y \to b_0$, $y \to b_1$, $y \to b_2$

- o/p lines → difference o/p D
→ Borrow o/p B → borrow o/p will be '1' if borrow is needed for (x-y) operation. otherwise borrow o/p will be '0'

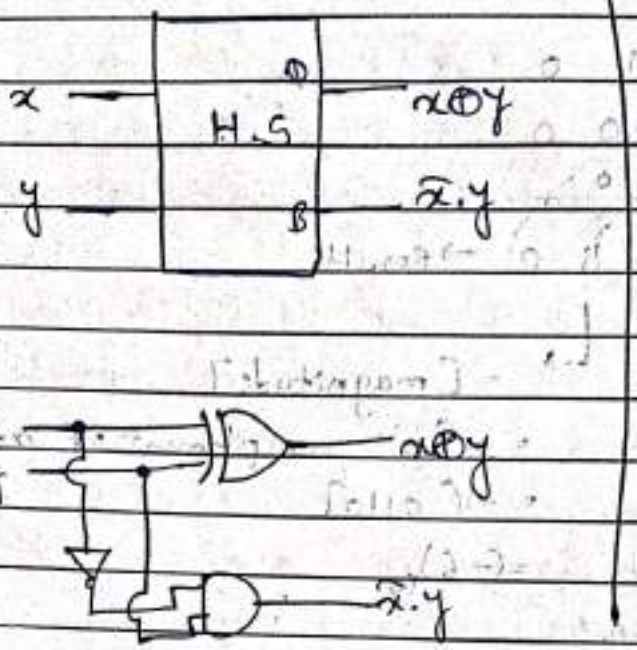
- Truth Table

I/p lines		o/p lines	
x	y	D	B
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

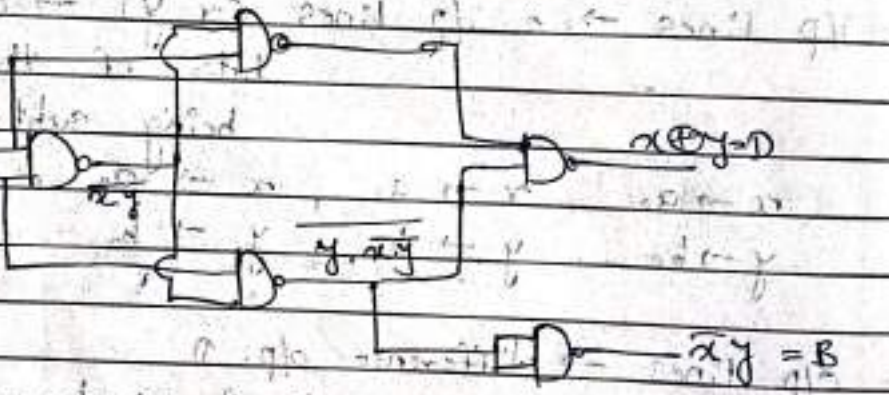
$D(x,y) = \Sigma(1,2)$
$= \bar{x}y + x\bar{y}$
$D = x \oplus y$
↳ Non-self Dual

$B(x,y) = \Sigma(1) = \bar{x}.y$
$B = \bar{x}.y$
↳ Non self Dual

Scanned with OKEN Scanner



- Using NAND Gates



- Note :- To implement half subtractor are required five (2 i/p NAND) or five (2 i/p NOR) gates are required

$A = a_3\ a_2\ a_1\ a_0$

$B = b_3\ b_2\ b_1\ b_0$

$B_3 \quad B_2 \quad B_1 \quad 0$

$B_4 \quad D_3 \quad D_2 \quad D_1 \quad D_0$

* Full Subtractor :-

- i/p lines → 3 i/p lines x, y, z → x, y → corresponding bits of the two numbers being subtracted ($x - y - z$)
  - $x \rightarrow a_i$
  - $y \rightarrow b_i$
  - $z \rightarrow B_i$ → Borrow o/p from just previous subtraction
  - → Borrow o/p from (a_i, b_i) position
- o/p lines → Difference o/p D
  - → Borrow o/p B → Borrow o/p will be '1' if borrow is needed for $(x-y-z)$ operation
  - otherwise borrow o/p is '0'

- Truth Table :-

Input lines			o/p lines	
x	y	z	D	B/A
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

$D(x,y,z) = \Sigma(1,2,4,7)$

$D = x \oplus y \oplus z$

→ Self-Dual

$B(x,y,z) = \Sigma(1,2,3,7)$

$= \bar{x}\bar{y}z + \bar{x}y\bar{z} + \bar{x}yz + xyz$

$B = \bar{x}y + yz + \bar{x}z$

- See the difference :- [Hidden Tricks]

① $x+y \rightarrow x.y$	$x+y+z \rightarrow xy + yz + xz$
$x-y \rightarrow \bar{x}y$	$x-y-z \rightarrow \bar{x}y + yz + \bar{x}z$
$y-x \rightarrow x\bar{y}$	$y-z-x \rightarrow x\bar{y} + \bar{y}z + xz$
	$z-x-y \rightarrow xy + y\bar{z} + x\bar{z}$

Scanned with OKEN Scanner

② $B(x,y,z) = \bar{x}\bar{y}z + \bar{x}y\bar{z} + \bar{x}yz + xyz$

$$\boxed{B(x,y,z) = \bar{x}y + z(\bar{x} \oplus y)}$$

$(x-y+z) \rightarrow B \rightarrow \bar{x}y + (\bar{x} \oplus y)z$

Here we are subtracting from x so therefore there is $\bar{x}$ is present

$y-z-x \rightarrow B \rightarrow \bar{y}z + (\bar{y} \oplus z)x$

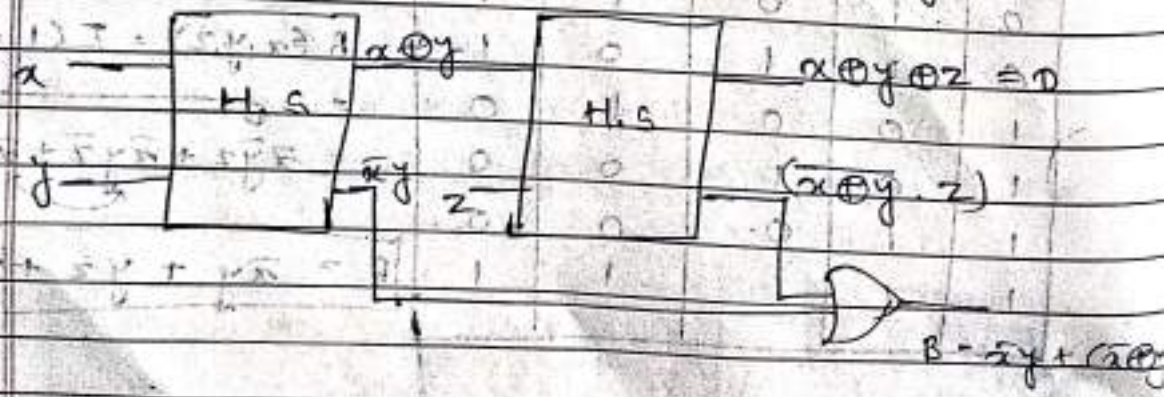
$z-x-y \rightarrow B \rightarrow \bar{z}x + (\bar{z} \oplus x)y$

③ $B = \bar{x}y + yz + \bar{x}z = \bar{x}y + [\bar{x} \oplus y]z$

$\bar{x}y \oplus yz \oplus \bar{x}z = \bar{x}y + yz + \bar{x}z$

$\bar{x}y \oplus (\bar{x} \oplus y)z = \bar{x}y + (\bar{x} \oplus y)z$

- Note: To implement F.S we require 9 [2 i/p NAND] or 9 [2 i/p NOR] gate
- Implementation using H.S and OR gate



- To subtract two n-bit numbers, number of H.S and number of OR gate required is

→ 1 H.S + (n-1) F.S

= 1 H.S + 2(n-1) H.S + (n-1) OR

= (2n-1) H.S + (n-1) OR

* K-Map (Karnaugh Map) :

- A K-Map is graphical method used in digital electronics and Boolean algebra to simplify logical expression.
- Used for Entries :
  1. '1'
  2. '0'
  3. 'x' → dont care
  4. Variable Entry

- Dont care :

In K-map, dont care condition are input combination for which the output is not important or never used in the actual circuit operation.

- If in given question A=1, B=1, should not be applied together then if we not applied that i/p combination then we don't need of the o/p of that combination, so that becomes dont care.

* BCD code : (Binary coded decimal) (8421 code)

Decimal	b_3	b_2	b_1	b_0	
0	0	0	0	0	Valid BCD code
1	0	0	0	1	
2	0	0	1	0	
3	0	0	1	1	
4	0	1	0	0	
5	0	1	0	1	
6	0	1	1	0	
7	0	1	1	1	
8	1	0	0	0	
9	1	0	0	1	
10	1	0	1	0	Invalid BCD code
11	1	0	1	1	
12	1	1	0	0	
13	1	1	0	1	
14	1	1	1	0	
15	1	1	1	1	

$(24)_{10} = (11000)_2 = (0010\ 0100)_{BCD}$

$(786)_{10} = (0111\ 1000\ 0110)_{BCD}$

	00	01	11	10
0				
1				

← These are in Gray code sequence and in gray code when we go in ascending order only one bit changes so that's why after 01 → 11 is coming

- 2-Variables, 3-variables and 4-variables K-map

f(A,B)

A \ B	0	1
$\bar{A}$ 0	0	1
A 1	2	3

, $f(A,B,C) = \bar{A}$ / A

	$\bar{B}\bar{C}$	$\bar{B}C$	BC	$B\bar{C}$
$\bar{A}$	0	1	3	2
A	4	5	7	6

$f(A,B,C,D)$

AB \ CD	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	0	1	3	2
$\bar{A}B$	4	5	7	6
AB	12	13	15	14
$A\bar{B}$	8	9	11	10

Imp

- How to select a part in k-map

① Always try to make the largest possible group
- group of 16 first
- then 8
- then 4
- then 2
- then 1

② In a group if one unused * 1 is present then that group is valid group and that always comes in output

③ In a group if all one's are used more than one then that group is redundant group that is not comming in o/p

④ If you want to solve unused 1 * and that unused 1 have more than 1 possible option pairs then prefer that option in which more unused 1's are comming

⑤ At first always give priority to solve that one's which are present at the edges

- Note :- K-Map may have more than one solution but it is not always necessary.
- Note :- When given funct^n of 4 variables then if in answer one variable is reduced then it is pair, if two variable is reduced then it is quad and if three variable is reduced then octal is produced

Que: $f(A,B,C) = A\bar{C} + A\bar{B}\bar{C} + BC$ Then minimize expression will be.

→ Now solving this numerical

① Method ① → convert it in canonical form then find numbers then put it into K-map and then solve

② Method ② → Without using above method we can directly put the values in the K-Map so, it is 3 variable function.

So, first $A\bar{C}$ means one variable is reduced so it is pair in A row and in which columns $\bar{C}$ is common, same as $A\bar{B}C$ → so it is separate function and BC so in BC column it is pair

	$\bar{B}\bar{C}$	$\bar{B}C$	BC	$B\bar{C}$
$\bar{A}$			1	
A	1	1	1	1

Now solve above K-Map

$= A + BC$

Que: $f(A,B,C) = \overline{\bar{A}C + B\bar{C}} + AB\bar{C}$, then minimized expression will be

→ In this expression, similarly for $\bar{A}C$ and $B\bar{C}$ mark in that boxes and put 1 in remaining all boxes because it's bar so as $AB\bar{C}$ find that box and put 1.

Now solve

	$\bar{B}\bar{C}$	$\bar{B}C$	BC	$B\bar{C}$
$\bar{A}$	1	.	.	.
A	1	1	1	1

$= A + \bar{B}\bar{C}$

* Variables K-Map

Que: A K-map is given

	$\bar{A}$	A
$\bar{B}$	$\bar{C}$	1
B		C

. The solution is

→ Now, first finding function with the help of above K-map

① Method 1 :-

$f(A,B,C) = \bar{A}\bar{B}\bar{C} + A\bar{B}1 + \bar{A}B0 + ABC$

$f(A,B,C) = \bar{A}\bar{B}\bar{C} + A\bar{B} + ABC$

Now put it into K-map

	$\bar{B}\bar{C}$	$\bar{B}C$	BC	$B\bar{C}$
$\bar{A}$	1			
A	1	1	1	1

$= \bar{B}\bar{C} + AC$

② Method - 2 :

	$\bar{A}$	A
$\bar{B}$	1	C
B	X	

→

	$\bar{A}$	A
$\bar{B}$	1	0
B	X	

Put C = 0

	$\bar{A}$	A
$\bar{B}$	1	1
B	X	

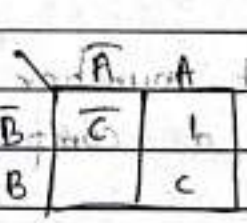
(or) Put C = 1

$\bar{C}\bar{A} + C\bar{B}$

$= \bar{C}\bar{A} + C\bar{B}$

- Note :- Here it is 3 variable function so possible values are only 2 (i.e. 0 & 1) so if the function is 4 variable then possible value funct^n is 4 (i.e. 00 & 01 & 11 & 10) so in that Equation we need to draw 4 K-map to solve the problem

- For variable K-map



$\rightarrow f(\bar{A}, \bar{B}, C) = \bar{C}A$
$f(\bar{A}, \bar{B}, C) = 1$
$f(\bar{A}, B, C) = 0$
$f(A, \bar{B}, C) = 0$
$f(A, B, C) = C$

- Saw Pdf No.: 06

* Comparator :-

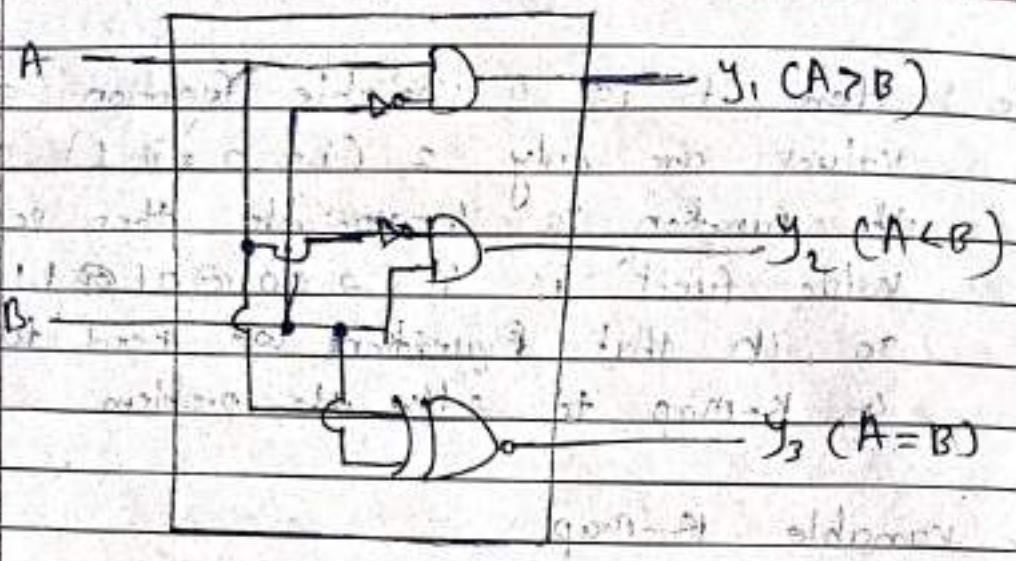


- A and B are single bit numbers then how to design the comparator

A	B	y_1 (A > B)	y_2 (A < B)	y_3 (A = B)
0	0	0	0	1
0	1	0	1	0
1	0	1	0	0
1	1	0	0	1

$y_1(A,B) = \Sigma(2) = A\bar{B}$
$y_2(A,B) = \Sigma(1) = \bar{A}B$
$y_3(A,B) = \Sigma(0,3) = \bar{A}\bar{B} + AB = A \odot B$

Scanned with OKEN Scanner



- When A and B are 3-bit number then how to write the output of a comparator

$A = a_2\ a_1\ a_0$

$B = b_2\ b_1\ b_0$

i/p lines ⇒ 6 i/p lines

- So in comparato logic is first we compar
- the MSB of both A and B (a_2 and b_2) and if $a_2 > b_2$ then $A > B$ but if $a_2 = b_2$ then goes to the second bit (a_1 and b_1) and so

- $y_1 (A>B) = (a_2 > b_2)$ or $(a_2 = b_2)$ & $(a_1 > b_1)$ or $(a_2 = b_2)$ & $(a_1 = b_1)$ & $(a_0 > b_0)$

  $= a_2 \cdot \overline{b_2} + (a_2 \odot b_2) \cdot a_1 \cdot \overline{b_1} + (a_2 \odot b_2)(a_1 \odot b_1) \cdot a_0 \cdot \overline{b_0}$

- $y_2 (A<B) = (a_2 < b_2)$ or $(a_2 = b_2)$ & $(a_1 < b_1)$ or $(a_2 = b_2)$ & $(a_1 = b_1)$ & $(a_0 < b_0)$

  $= \overline{a_2} b_2 + (a_2 \odot b_2) \cdot \overline{a_1} \cdot b_1 + (a_2 \odot b_2)(a_1 \odot b_1) \cdot \overline{a_0} \cdot b_0$

- $y_3 (A=B) = (a_2 = b_2)$ & $(a_1 = b_1)$ & $(a_0 = b_0)$

  $= (a_2 \odot b_2) \cdot (a_1 \odot b_1) \cdot (a_0 \odot b_0)$

Scanned with OKEN Scanner

max term → o/p is false

- Now Most Important
  - If A and B both have same bits
    A → n bits
    B → n bits
    Then,

① $N_1 (A>B) = N_2 (A<B)$
$= 0 + 1 + 2 + 3 + \ldots + (2^n - 1)$

② $N_3 (A=B) = 2^n$

$N_1 + N_2 + N_3 = 2^{2n}$ → Total combinations (A+B)

- If A has 2 bits then its combination $2^n = 2^2 = 4$

- Now,
  - If A has different bits than B
    A → n_1 bits
    B → n_2 bits
    Total combinations → $2^{(n_1+n_2)}$

- Case I $(n_1 < n_2)$
  $N_1 (A>B) = 0 + 1 + 2 + \ldots + (2^{n_1} - 1)$
  $N_2 (A<B) = 2^{(n_1+n_2)} - (N_1 + N_3)$
  $N_3 (A=B) = 2^{n_1}$

- Case II $(n_1 > n_2)$
  $N_1 (A>B) = 2^{(n_1+n_2)} - (N_2 + N_3)$
  $N_2 (A<B) = 0 + 1 + 2 + \ldots + (2^{n_2} - 1)$
  $N_3 (A=B) = 2^{n_2}$

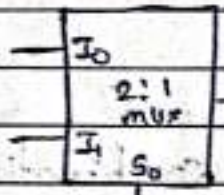
Scanned with OKEN Scanner

* MUX :-

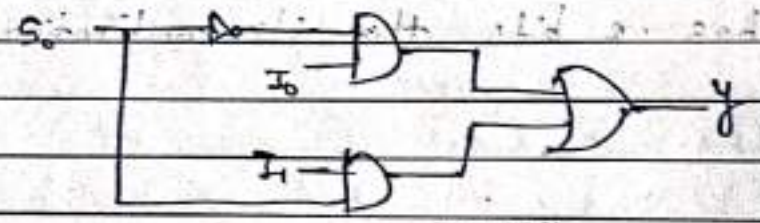
* What is MUX ?

↳ It is combinational circuit having many i/ps and one o/p line and on the basis of select i/p line, one of the i/p is transfered to o/p line

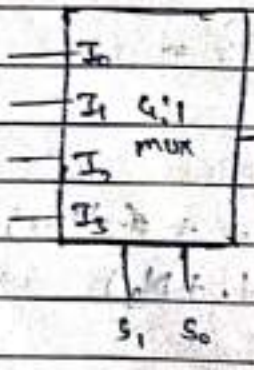
① 2:1 MUX



$$y = (\bar{S}_0 I_0 + S_0 I_1)$$



② 4:1 MUX



$$y = \bar{S}_1 \bar{S}_0 I_0 + \bar{S}_1 S_0 I_1 + S_1 \bar{S}_0 I_2 + S_1 S_0 I_3$$

③ • Relation Between number of i/p lines M and number of select lines N for functioning of MUX

$$2^N \geq M$$

$$\log_2 2^N \geq \log_2 M$$

$$N \geq \log_2 M$$

Scanned with OKEN Scanner

- Higher order MUX using lower order MUX

① $4:1 \xrightarrow{\text{Using } 2:1 \text{ MUX}} \frac{4}{2} + \frac{2}{2} + 1 \longrightarrow 2+1 = 3$

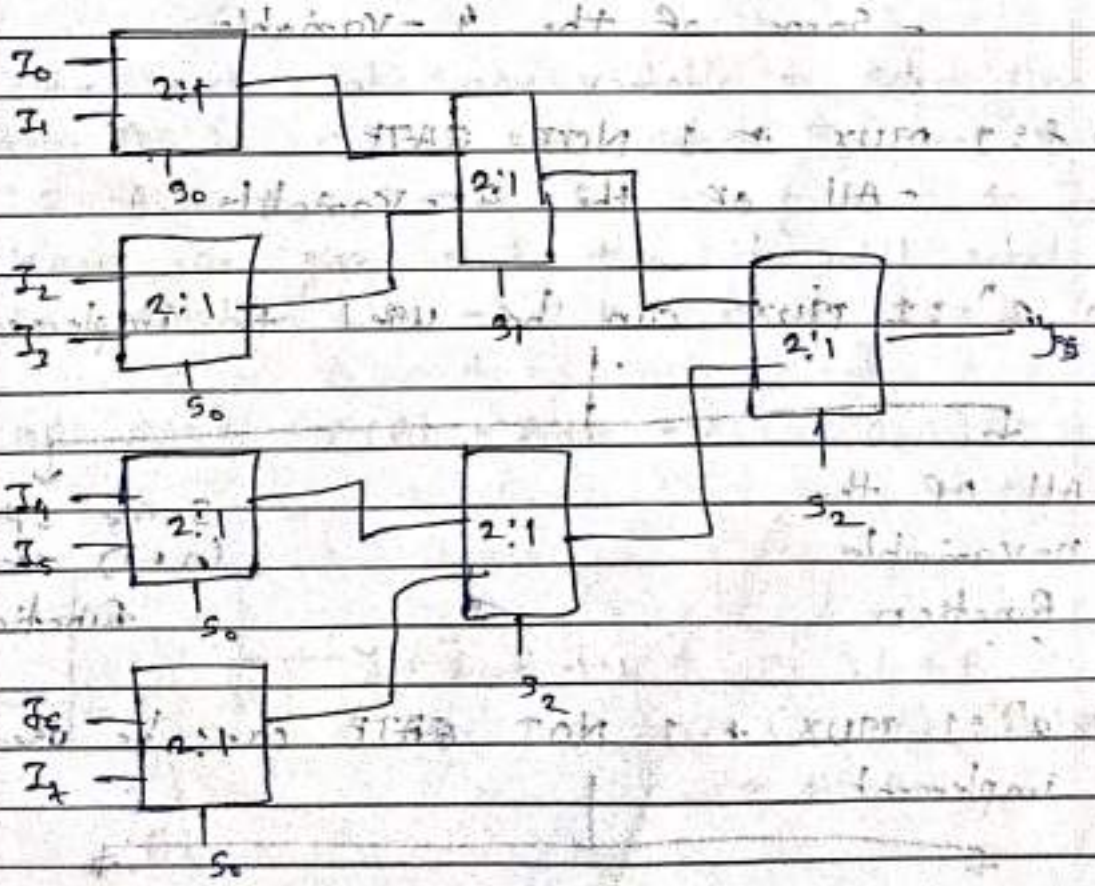
② $8:1 \xrightarrow{2:1 \text{ MUX}} 4+2+1 = 7 = (2^3 - 1)$

③ $16:1 \xrightarrow{2:1} 8+4+2+1 = 15 = (2^4 - 1)$

④ $2^n:1 \xrightarrow{2:1} (2^n - 1)$

- Implement 8:1 MUX using 2:1 MUX and design the complete circuit

$8:1 \xrightarrow{2:1} 4+2+1 = 7$ $I_0 \dots I_7$ } available
S_0, S_1, S_2



DATE

Very good method is given by Sanu because to solve the problem

Sanu pdf 09 → in this last numerical imp & its sol^n

① 4:1 MUX can be used to implement
- All 2-variable function
- Some of the 3-variable function

Reason → When we solve logical function of SOP form using lower order MUX at that time at the i/p line (0, 1, A, $\bar{A}$) like are coming so for $\bar{A}$ it needs one NOT Gate

② 4:1 MUX + 1 NOT GATE used to implement
- All of 3-variable function

③ 8:1 MUX can be used to implement
- All of the 3-variable function
- Some of the 4-variable

④ 8:1 MUX + 1 NOT GATE
- All of the 4-variable

⑤ 2^n:1 MUX can be used to implement

All of the n-variable function

Some of the (n+1) variable functions

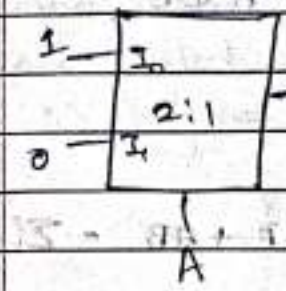
⑥ 2^n:1 MUX + 1 NOT GATE can be used to implement

All of the (n+1) variable function

Scanned with OKEN Scanner

* MUX as a Universal Circuit

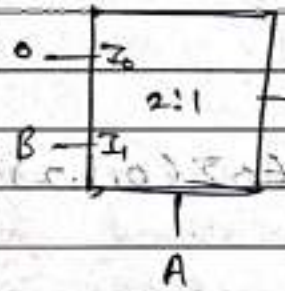
- NOT GATE : $F(A) = \bar{A} = \Sigma(0) = \Pi(1)$



$Y = \bar{A} \cdot 1 + A \cdot 0$

$= \bar{A}$

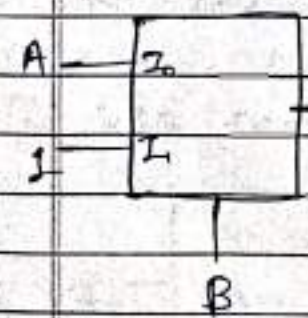
- AND GATE : $F(A) \rightarrow A \cdot B = \Sigma(3) = \Pi(0,1,2)$



$Y = \bar{A} \cdot 0 + AB = A \cdot B$

- Here first take any variable to select line now if A is 0 it will select I_0 and in function if $AB \rightarrow 0 \cdot B \rightarrow 0$ so we give 0 to I_0 Similarly now we give $A = 1$ then it will select I_1 and in function $AB \rightarrow 1 \cdot B \rightarrow B$ so $I_1 = B$

- OR GATE : $F(A) = A + B = \Sigma(1,2,3) = \Pi(0)$



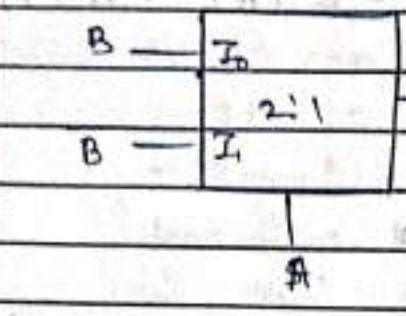
$Y = \bar{B} \cdot A + B \cdot 1 = \bar{B}A + B$

$(\bar{B} + B)(A + B)$

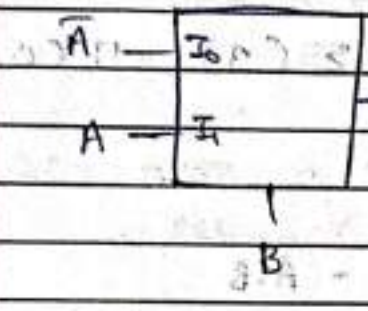
$= A + B$

Scanned with OKEN Scanner

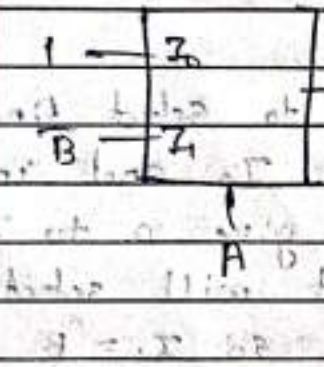
⑤ XOR GATE : $F(A) = A \oplus B = \bar{A}B + A\bar{B} = \Sigma(1,2)$



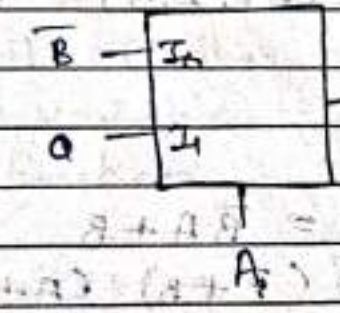
⑥ XNOR GATE : $F(A) = A \odot B = \bar{A}\bar{B} + AB = \Sigma(0,3)$



⑦ NAND GATE : $F(A) = \overline{A.B} = \Sigma(0,1,2) = \Pi(3)$



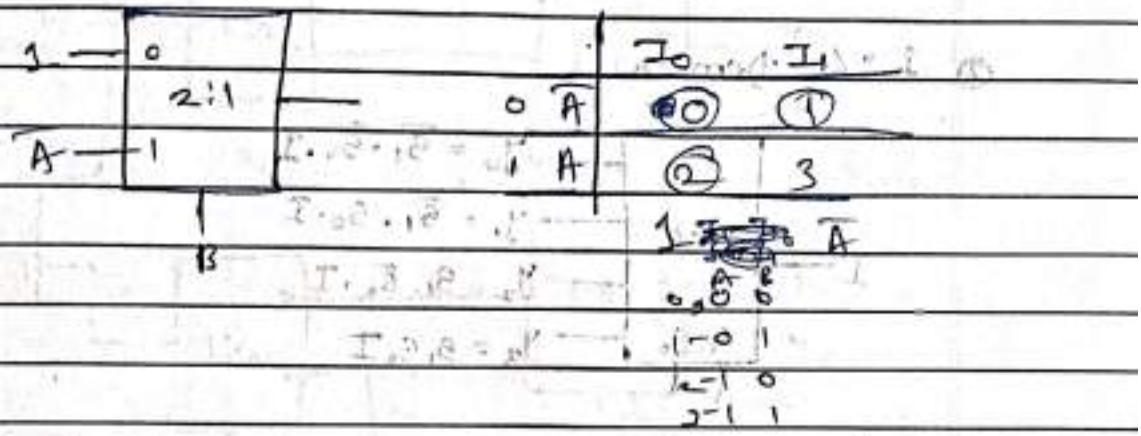
⑧ NOR GATE : $F(A) = \overline{A+B} = \Sigma(0)$



Scanned with OKEN Scanner

eg $F(A,B) = \Sigma(0,1,2)$ → implement this logical function using 2:1 MUX

→ So, here in function two variable is given and use 2:1 MUX but in 2:1 MUX only one select line is present so other variable is adjusted on the i/p lines.

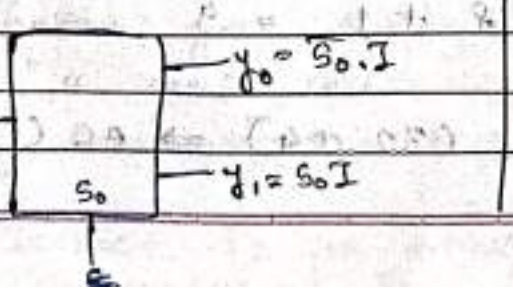


- Note :-
- To implement basic gates [NOT, AND, OR] we require one [2:1] MUX for each gate
- To implement Non-basic gates [XOR, XNOR, NAND, NOR] we require two [2:1] MUX for each gate

Demux :-

- It is combinational ckt having one i/p and many o/p lines. And on the basis of select i/p lines, i/p line is transferred to one of the o/p lines.

① 1:2 Demux



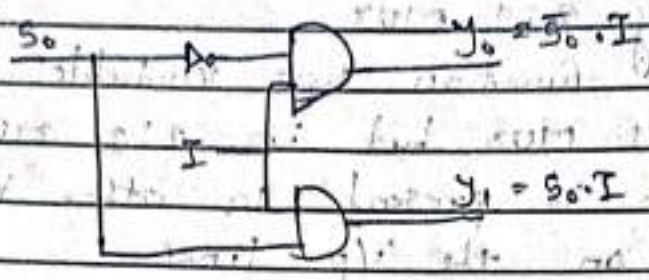
$y_0 = 0 \rightarrow y_0 = 1.I = I$

$y_1 = 0.I = 0$

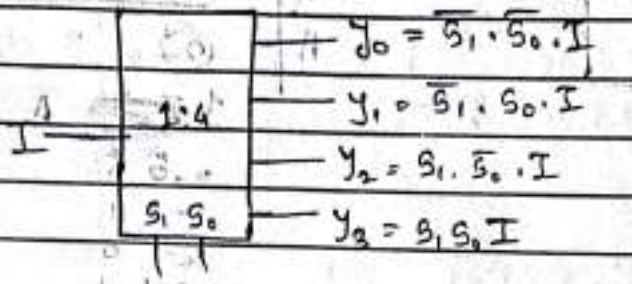
$S_0 = 1 \rightarrow y_0 = 0$

$y_1 = I$

Internal Circuit



② 1:4 Demux



- Higher order DEMUX using lower order Demux

① 1:4 $\xrightarrow[1:2]{\text{using}}$ $2+1 = 3 \Rightarrow (2^2-1)$

② 1:8 $\xrightarrow[1:2]{\text{using}}$ $4+2+1 = 7 \Rightarrow (2^3-1)$

③ 1:16 $\xrightarrow[1:2]{\text{using}}$ $8+4+2+1 = 15 \Rightarrow (2^4-1)$

④ 1:2^n $\xrightarrow[1:2]{\text{using}}$ (2^n-1)

⑤ 1:16 $\xrightarrow[1:4]{\text{using}}$ $4+1 = 5$

⑥ 1:32 $\xrightarrow[1:4]{\text{using}}$ $8+2 \Rightarrow 10\,(1:4) + 1\,(1:2)$

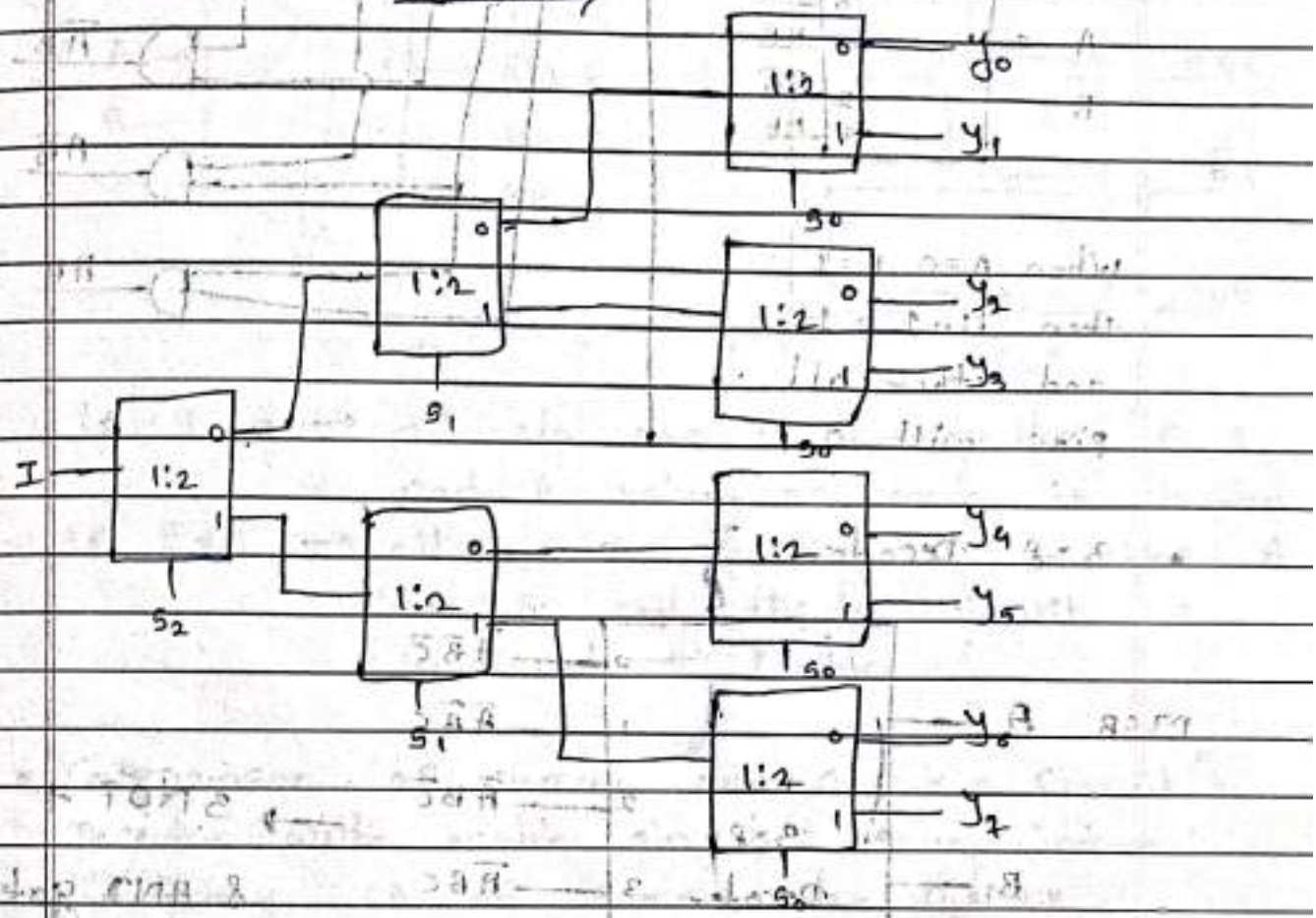
⑦ 1:64 $\xrightarrow[1:8]{\text{using}}$ $8+1 = 9$

⑧ 1:256 $\xrightarrow[1:8]{\text{using}}$ $(32+4) \Rightarrow 36\,(1:4) + 1\,(1:2)$

Scanned with OKEN Scanner

eg. 1:8 $\xrightarrow{1:2}$ 4+2+1

I $\rightleftharpoons$ I, S_2, S_1, S_0



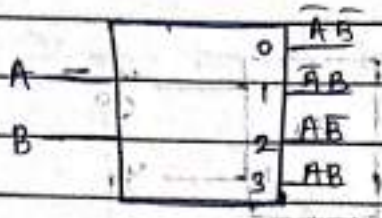
* Decoder :-
- It is combinational ckt having many i/p lines and many o/p lines
- It is used to convert binary into other code
  - eg. 3:8 decoder → binary to octal
  - 4:16 decoder → binary to hexadecimal
- $n : 2^n$ → Order

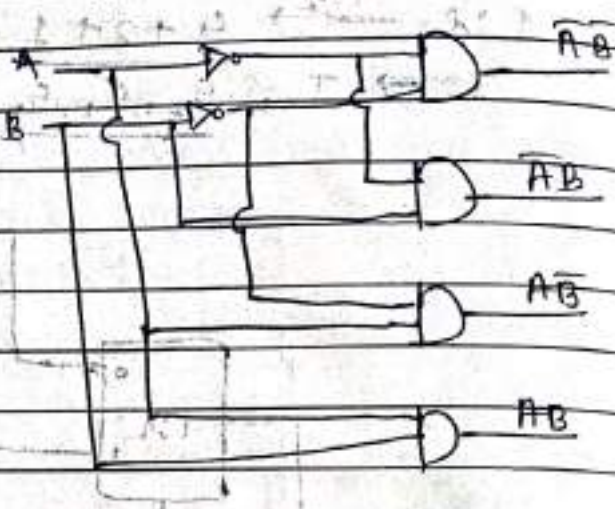
i/p o/p

- Whenever you apply any one combination to i/p pins then corresponding o/p pin of the combination is '1' and all other are '0'
- For Decoder i/p is never be zero because any one

Internal circuit

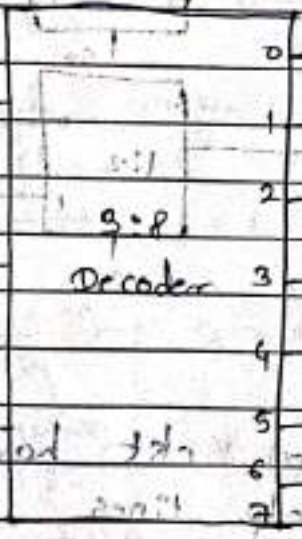
- 2:4 decoder





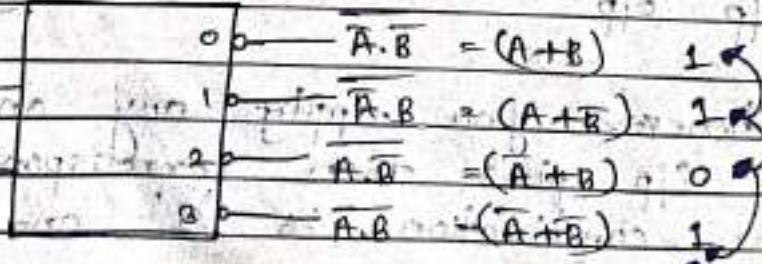
When A=0, B=1 then Pin 1 = 1 and other all pins will 0

- 3:8 Decoder



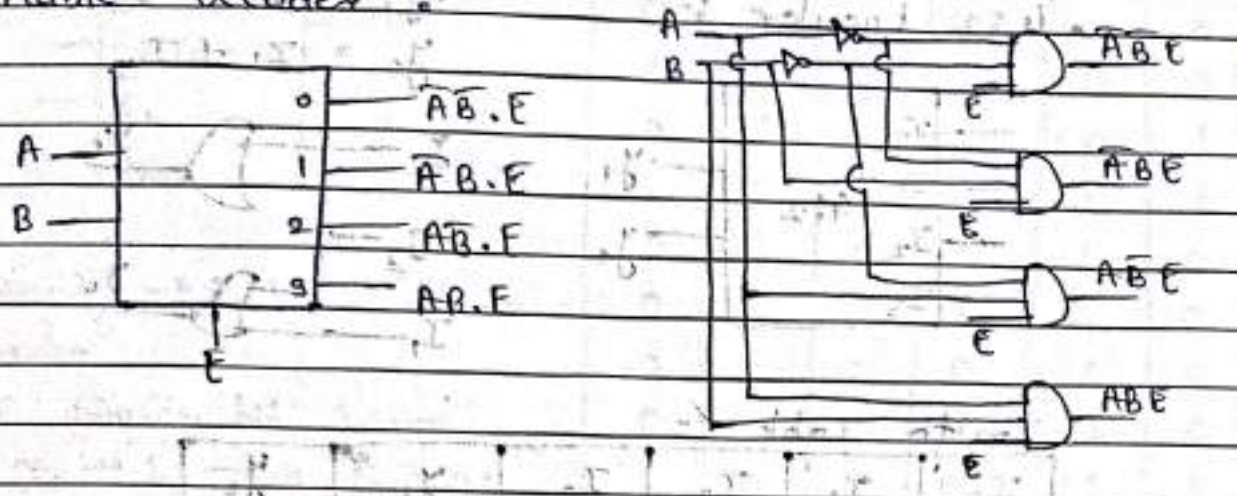
→ 3 NOT + 8 AND gate (3-i/p)

- Low Enable Decoder :



When we apply A=1, B=0

• Enable Decoder :-
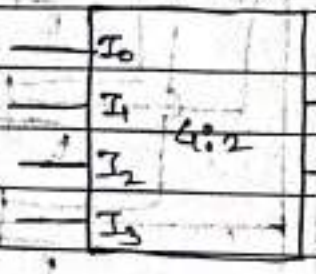


if E=0 → all o/p are 0 irrespective of A and B values → Decoder is disabled.
if E=1 → all o/p are function of variables A and B and decoder circuit is functional → Enable.

• Comparison of Demux and Decoder Circuit :-
- Decoder with the enable pin has same internal circuitery as of same order DEMUX

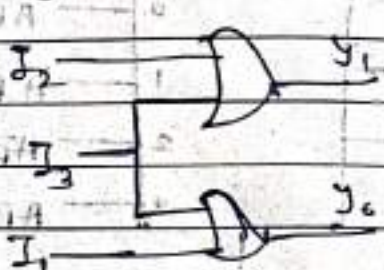
• Encoder :-
- It is combinational ckt having many i/p lines and many o/p lines
- Used to convert other codes into binary
eg. 8:3 Encoder → Octal to Binary
16:4 Encoder → Hexadecimal to Binary
- 2^n in → order of Encoder
- Whenever we give i/p to i/p line of Encoder at that time only one (any) i/p is one and all other are zero
- For Encoder i/p never be zero because any one bit is required to enable the encoder

- 4:2 Encoder :-



$y_1 = I_2 + I_3$

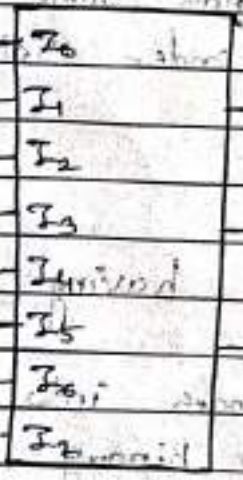
$y_0 = I_1 + I_3$



Truth Table :-

I_0	I_1	I_2	I_3	y_1	y_0
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1

- 8:3 Encoder :-



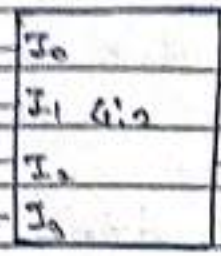
don't care condition $= 2^8 - 8 = 248$

$y_2 = I_4 + I_5 + I_6 + I_7$

$y_1 = I_2 + I_3 + I_6 + I_7$

$y_0 = I_1 + I_3 + I_5 + I_7$

- Priority Encoder :-
- It is also same as encoder but here o/p is calculated by using priority bit
- Here we can apply all possible combination in which also having more than 1 in I's in combination. But o/p is select the priority bit and gives the o/p.



- Priority → $I_3 > I_2 > I_1 > I_0$ order
- If priority bit in →
  - $I_0 \rightarrow 1 \rightarrow y_1 y_0 = 00$
  - $I_1 \rightarrow 1 \rightarrow y_1 y_0 = 01$
  - $I_2 \rightarrow 1 \rightarrow y_1 y_0 = 10$
  - $I_3 \rightarrow 1 \rightarrow y_1 y_0 = 11$

I_3	I_2	I_1	I_0	y_1	y_0
0	0	0	0	x	x
0	0	0	1	0	0
0	0	1	0	0	1
0	0	1	1	0	1
0	1	0	0	1	0
0	1	0	1	1	0
0	1	1	0	1	0
0	1	1	1	1	0
1	0	0	0	1	1
1	0	0	1	1	1
1	0	1	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1
1	1	0	1	1	1
1	1	1	0	1	1
1	1	1	1	1	1

$y_1 [I_3, I_2, I_1, I_0] = \Sigma(4, 5, 6, 7, \ldots\ldots 15) + d\Sigma(0)$

$y_0 [I_3, I_2, I_1, I_0] = \Sigma(2, 3, 8, 9, 10, 11, 12, 13, 14, 15) + d\Sigma(0)$

on solving above eqⁿ using k-map

$$y_1 = (I_2 + I_3)$$
$$y_0 = I_3 + I_1 \bar{I}_2$$

Scanned with OKEN Scanner